```
            }
            du=funcd.df(u);                          Now all the housekeeping, sigh.
            if (fu <= fx) {
                if (u >= x) a=x; else b=x;
                mov3(v,fv,dv,w,fw,dw);
                mov3(w,fw,dw,x,fx,dx);
                mov3(x,fx,dx,u,fu,du);
            } else {
                if (u < x) a=u; else b=u;
                if (fu <= fw || w == x) {
                    mov3(v,fv,dv,w,fw,dw);
                    mov3(w,fw,dw,u,fu,du);
                } else if (fu < fv || v == x || v == w) {
                    mov3(v,fv,dv,u,fu,du);
                }
            }
        }
        throw("Too many iterations in routine dbrent");
    }
};
```

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 55; 454–458.[1]

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), p. 78.

10.5 Downhill Simplex Method in Multidimensions

With this section we begin consideration of multidimensional minimization, that is, finding the minimum of a function of more than one independent variable. This section stands apart from those that follow, however: All of the algorithms after this section will make explicit use of a one-dimensional minimization algorithm as a part of their computational strategy. This section implements an entirely self-contained strategy, in which one-dimensional minimization does not figure.

The *downhill simplex method* is due to Nelder and Mead [1]. The method requires only function evaluations, not derivatives. It is not very efficient in terms of the number of function evaluations that it requires. Powell's method (§10.7) or the DFP method with finite differences (§10.9) is almost surely faster in all likely applications. However, the downhill simplex method may frequently be the *best* method to use if the figure of merit is "get something working quickly" for a problem whose computational burden is small.

The method has a geometrical naturalness about it that makes it delightful to describe or work through:

A *simplex* is the geometrical figure consisting, in N dimensions, of $N+1$ points (or vertices) and all their interconnecting line segments, polygonal faces, etc. In two dimensions, a simplex is a triangle. In three dimensions, it is a tetrahedron, not necessarily the regular tetrahedron. (The *simplex method* of linear programming, described in §10.10, also makes use of the geometrical concept of a simplex. Otherwise

it is completely unrelated to the algorithm that we are describing in this section.) In general we are only interested in simplexes that are nondegenerate, i.e., that enclose a finite inner N-dimensional volume. If any point of a nondegenerate simplex is taken as the origin, then the N other points define vector directions that span the N-dimensional vector space.

In one-dimensional minimization, it was possible to bracket a minimum, so that the success of a subsequent isolation was guaranteed. Alas! There is no analogous procedure in multidimensional space. For multidimensional minimization, the best we can do is give our algorithm a starting guess, that is, an N-vector of independent variables as the first point to try. The algorithm is then supposed to make its own way downhill through the unimaginable complexity of an N-dimensional topography, until it encounters a (local, at least) minimum.

The downhill simplex method must be started not just with a single point, but with $N + 1$ points, defining an initial simplex. If you think of one of these points (it matters not which) as being your initial starting point $\mathbf{P}_0$, then you can take the other N points to be

$$\mathbf{P}_i = \mathbf{P}_0 + \Delta \mathbf{e}_i \tag{10.5.1}$$

where the $\mathbf{e}_i$'s are N unit vectors, and where Δ is a constant that is your guess of the problem's characteristic length scale. (Or, you could have different Δ_i's for each vector direction.)

The downhill simplex method now takes a series of steps, most steps just moving the point of the simplex where the function is largest ("highest point") through the opposite face of the simplex to a lower point. These steps are called reflections, and they are constructed to conserve the volume of the simplex (and hence maintain its nondegeneracy). When it can do so, the method expands the simplex in one or another direction to take larger steps. When it reaches a "valley floor," the method contracts itself in the transverse direction and tries to ooze down the valley. If there is a situation where the simplex is trying to "pass through the eye of a needle," it contracts itself in all directions, pulling itself in around its lowest (best) point. The routine name `amoeba` is intended to be descriptive of this kind of behavior; the basic moves are summarized in Figure 10.5.1.

Termination criteria can be delicate in any multidimensional minimization routine. Without bracketing, and with more than one independent variable, we no longer have the option of requiring a certain tolerance for a single independent variable. We typically can identify one "cycle" or "step" of our multidimensional algorithm. It is then possible to terminate when the vector distance moved in that step is fractionally smaller in magnitude than some tolerance `tol`. Alternatively, we could require that the decrease in the function value in the terminating step be fractionally smaller than some tolerance `ftol`. Note that while `tol` should not usually be smaller than the square root of the machine precision, it is perfectly appropriate to let `ftol` be of order the machine precision (or perhaps slightly larger so as not to be confused by roundoff).

Note well that either of the above criteria might be fooled by a single anomalous step that, for one reason or another, failed to get anywhere. Therefore, it is frequently a good idea to *restart* a multidimensional minimization routine at a point where it claims to have found a minimum. For this restart, you should reinitialize any ancillary input quantities. In the downhill simplex method, for example, you should reinitialize N of the $N + 1$ vertices of the simplex again by equation (10.5.1), with

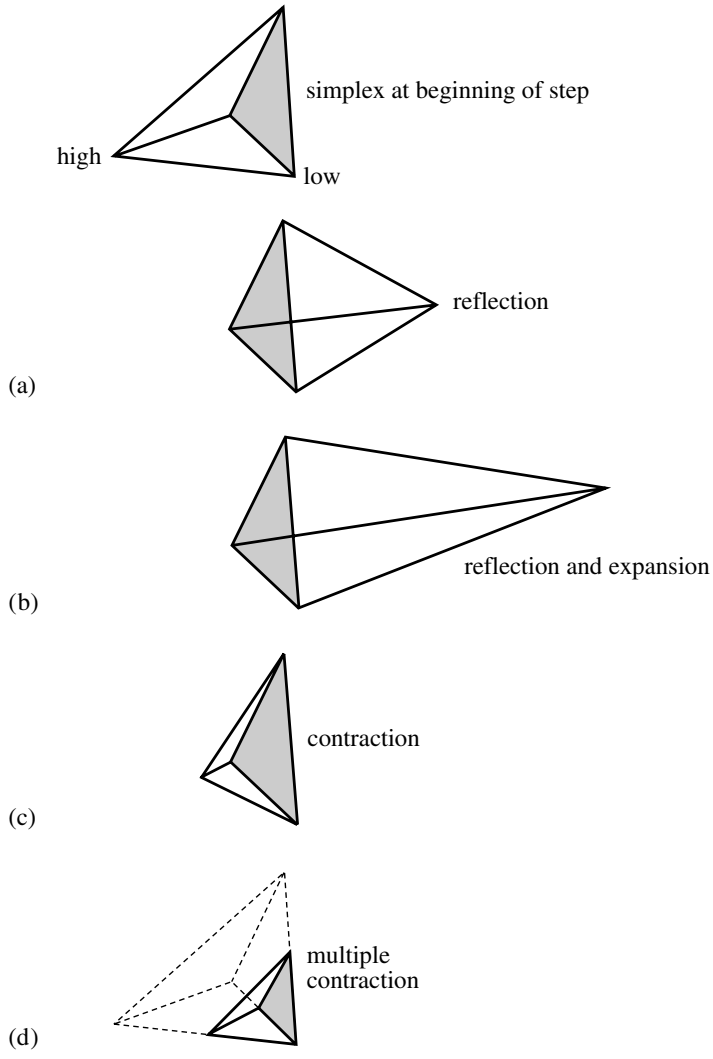

Figure 10.5.1. Possible outcomes for a step in the downhill simplex method. The simplex at the beginning of the step, here a tetrahedron, is shown, top. The simplex at the end of the step can be any one of (a) a reflection away from the high point, (b) a reflection and expansion away from the high point, (c) a contraction along one dimension from the high point, or (d) a contraction along all dimensions toward the low point. An appropriate sequence of such steps will always converge to a minimum of the function.

$\mathbf{P}_0$ being one of the vertices of the claimed minimum.

Restarts should never be very expensive; your algorithm did, after all, converge to the restart point once, and now you are starting the algorithm already there.

The routine below has three different user interfaces. The simplest requires you to supply the initial simplex as in equation (10.5.1):

```
Amoeba am(ftol);
VecDoub point = ...; Doub del = ...;
pmin=am.minimize(point,del,func);
```

The value of the function at the minimum is available in `am.fmin`.

Second, you can use equation (10.5.1) with a vector of increments Δ_i:

```
VecDoub dels = ...;
pmin=am.minimize(point,dels,func);
```

Lastly, you can provide the simplex as an $(N+1) \times N$ matrix whose rows are the coordinates of each vertex:

```
MatDoub p = ...;
pmin=am.minimize(p,func);
```

Consider, then, our N-dimensional amoeba:

amoeba.h

```
struct Amoeba {
```

Multidimensional minimization by the downhill simplex method of Nelder and Mead.

```
    const Doub ftol;
    Int nfunc;                          The number of function evaluations.
    Int mpts;
    Int ndim;
    Doub fmin;                          Function value at the minimum.
    VecDoub y;                          Function values at the vertices of the simplex.
    MatDoub p;                          Current simplex.
    Amoeba(const Doub ftoll) : ftol(ftoll) {}
```

The constructor argument `ftoll` is the fractional convergence tolerance to be achieved in the function value (n.b.!).

```
    template <class T>
    VecDoub minimize(VecDoub_I &point, const Doub del, T &func)
```

Multidimensional minimization of the function or functor `func(x)`, where `x[0..ndim-1]` is a vector in `ndim` dimensions, by the downhill simplex method of Nelder and Mead. The initial simplex is specified as in equation (10.5.1) by a `point[0..ndim-1]` and a constant displacement `del` along each coordinate direction. Returned is the location of the minimum.

```
    {
        VecDoub dels(point.size(),del);
        return minimize(point,dels,func);
    }
    template <class T>
    VecDoub minimize(VecDoub_I &point, VecDoub_I &dels, T &func)
```

Alternative interface that takes different displacements `dels[0..ndim-1]` in different directions for the initial simplex.

```
    {
        Int ndim=point.size();
        MatDoub pp(ndim+1,ndim);
        for (Int i=0;i<ndim+1;i++) {
            for (Int j=0;j<ndim;j++)
                pp[i][j]=point[j];
            if (i !=0 ) pp[i][i-1] += dels[i-1];
        }
        return minimize(pp,func);
    }
    template <class T>
    VecDoub minimize(MatDoub_I &pp, T &func)
```

Most general interface: initial simplex specified by the matrix `pp[0..ndim][0..ndim-1]`. Its `ndim+1` rows are `ndim`-dimensional vectors that are the vertices of the starting simplex.

```
    {
        const Int NMAX=5000;            Maximum allowed number of function evaluations.
        const Doub TINY=1.0e-10;
        Int ihi,ilo,inhi;
        mpts=pp.nrows();
        ndim=pp.ncols();
        VecDoub psum(ndim),pmin(ndim),x(ndim);
        p=pp;
        y.resize(mpts);
        for (Int i=0;i<mpts;i++) {
            for (Int j=0;j<ndim;j++)
                x[j]=p[i][j];
            y[i]=func(x);
```

```
    }
    nfunc=0;
    get_psum(p,psum);
    for (;;) {
        ilo=0;
```

First we must determine which point is the highest (worst), next-highest, and lowest (best), by looping over the points in the simplex.

```
        ihi = y[0]>y[1] ? (inhi=1,0) : (inhi=0,1);
        for (Int i=0;i<mpts;i++) {
            if (y[i] <= y[ilo]) ilo=i;
            if (y[i] > y[ihi]) {
                inhi=ihi;
                ihi=i;
            } else if (y[i] > y[inhi] && i != ihi) inhi=i;
        }
        Doub rtol=2.0*abs(y[ihi]-y[ilo])/(abs(y[ihi])+abs(y[ilo])+TINY);
```

Compute the fractional range from highest to lowest and return if satisfactory.

```
        if (rtol < ftol) {          If returning, put best point and value in slot 0.
            SWAP(y[0],y[ilo]);
            for (Int i=0;i<ndim;i++) {
                SWAP(p[0][i],p[ilo][i]);
                pmin[i]=p[0][i];
            }
            fmin=y[0];
            return pmin;
        }
        if (nfunc >= NMAX) throw("NMAX exceeded");
        nfunc += 2;
```

Begin a new iteration. First extrapolate by a factor -1 through the face of the simplex across from the high point, i.e., reflect the simplex from the high point.

```
        Doub ytry=amotry(p,y,psum,ihi,-1.0,func);
        if (ytry <= y[ilo])
```

Gives a result better than the best point, so try an additional extrapolation by a factor 2.

```
            ytry=amotry(p,y,psum,ihi,2.0,func);
        else if (ytry >= y[inhi]) {
```

The reflected point is worse than the second-highest, so look for an intermediate lower point, i.e., do a one-dimensional contraction.

```
            Doub ysave=y[ihi];
            ytry=amotry(p,y,psum,ihi,0.5,func);
            if (ytry >= ysave) {        Can't seem to get rid of that high point.
                for (Int i=0;i<mpts;i++) {     Better contract around the lowest
                    if (i != ilo) {            (best) point.
                        for (Int j=0;j<ndim;j++)
                            p[i][j]=psum[j]=0.5*(p[i][j]+p[ilo][j]);
                        y[i]=func(psum);
                    }
                }
                nfunc += ndim;          Keep track of function evaluations.
                get_psum(p,psum);       Recompute psum.
            }
        } else --nfunc;                 Correct the evaluation count.
    }                                   Go back for the test of doneness and the next
}                                           iteration.
inline void get_psum(MatDoub_I &p, VecDoub_O &psum)
```

Utility function.

```
{
    for (Int j=0;j<ndim;j++) {
        Doub sum=0.0;
        for (Int i=0;i<mpts;i++)
            sum += p[i][j];
        psum[j]=sum;
    }
}
```